# Summary and Quiz

Explore the core AWS database services such as RDS, Aurora, and DynamoDB, including caching and scaling strategies. Learn about high availability, data consistency, and performance optimization. Practice your knowledge with a quiz that reinforces key concepts for cloud developers.

In this lesson, we'll summarize what we have learned about database services offered by AWS. Also, we'll test our knowledge through a quiz.

## Summary

Let's quickly summarize what we have learned in the Database module:

### Relational Database Service (RDS)

Amazon RDS simplifies relational database management with automated scaling, backup, and monitoring.

- It supports popular database engines like PostgreSQL, MySQL, and Aurora.
- Developers choose instance classes, types, and EBS storage options based on workload needs.
- Deployment options include Single-AZ, Multi-AZ Instance, and Multi-AZ DB Cluster for varying availability and scalability.
- Multi-AZ DB Clusters enable automatic failover and read scalability with semi-synchronous replication.

Ask

### Amazon Aurora

Amazon Aurora is a cloud-optimized relational database service compatible with MySQL and PostgreSQL.

- Aurora clusters feature fault-tolerant, multi-AZ distributed storage with independent compute scaling.
- High availability is built-in, with automatic failover among Aurora Replicas across AZs.
- Aurora global databases enable low-latency reads and disaster recovery across multiple AWS regions.
- Automated backups and manual snapshots offer flexible data recovery and cloning options.
- Aurora Serverless v2 provides auto-scaling database capacity for variable and unpredictable workloads.
- Aurora's architecture reduces replication lag and downtime, ensuring minimal impact during failover events.

## RDS Proxy

Amazon RDS Proxy efficiently manages database connections by pooling and reusing them for applications.

- It improves database performance under high connection loads and stabilizes applications during failovers.
- RDS Proxy supports secure authentication using AWS Secrets Manager and IAM authentication tokens.
- It reduces overhead for Lambda, containerized apps, and small-instance workloads by minimizing direct connection handling.
- TLS/SSL encryption is enforced by default to secure all client-proxy-database traffic.

Ask

## DynamoDB

DynamoDB is a fully managed NoSQL database offering millisecond performance, flexible schemas, and automatic scaling.

- Each table has a mandatory primary key—a simple partition key or a composite key with a partition and sort key.
- DynamoDB supports schemaless design for attributes but demands disciplined consistency for keys and indexes.
- Persistent storage across multiple AZs ensures high durability and availability without manual backups.
- DynamoDB Accelerator (DAX) provides microsecond latency caching for fast retrieval of eventually consistent data.
- DynamoDB supports both eventually consistent and strongly consistent reads, plus full ACID transaction capabilities.

## Performance optimization of DynamoDB

DynamoDB supports provisioned and on-demand capacity modes to suit predictable or unpredictable workloads.

- Provisioned mode allows setting RCUs and WCUs manually, while on-demand mode auto-scales and charges per request.
- Internal mechanisms like burst capacity, adaptive capacity, and warm throughput maintain low-latency performance.
- High-cardinality partition keys are critical for evenly distributing traffic and avoiding hot partitions.
- DAX caching, random key sharding, and backoff strategies help mitigate hot partition issues during traffic spikes.

## Data retrieval in DynamoDB

Ask

- Queries retrieve items by partition key and are highly efficient, while scans read the entire table and consume more capacity.

- Filters refine results logically after data retrieval; projections limit returned attributes to optimize payload size.
- Local Secondary Indexes (LSIs) allow alternative sorting for the same partition key and must be defined at table creation.
- Global Secondary Indexes (GSIs) enable flexible new access patterns with different partition keys and can be added later.
- Index design should prioritize minimal projections, sparse indexing, and monitor impact to optimize cost and performance.

## DynamoDB streams

DynamoDB Streams capture real-time, ordered changes to table data for event-driven application workflows.

- Streams offer multiple view types (`KEYS_ONLY`, `NEW_IMAGE`, `OLD_IMAGE`, and `NEW_AND_OLD_IMAGES`) to fit different use cases.
- Integrating Streams with AWS Lambda enables serverless reactions to database events without constant polling.
- Choosing the right stream view is crucial. For example, `NEW_IMAGE` is ideal when the full new data is needed for notifications.

## DynamoDB transactions and locking

DynamoDB supports ACID transactions using `TransactWriteItems` and `TransactGetItems` APIs across one or more tables.

- `TransactWriteItems` allows multiple write operations to succeed or fail atomically, preventing partial updates.
- Conditional expressions and idempotency tokens help manage concurrency and retry safely in high-traffic systems.
- `TransactGetItems` ensures consistent, atomic multi-item reads, critical for applications needing point-in-time accuracy.

Ask

- DynamoDB uses lightweight optimistic locking for transactions, avoiding manual lock management by developers.

## Caching strategies

Caching improves performance by storing frequently accessed data closer to the application, reducing database load and latency.

- Write-through caching ensures consistency by updating both cache and database simultaneously, but slightly slows down writes.
- Read-through and lazy loading (cache-aside) fetch and cache data on demand, optimizing cache usage for frequently accessed items.
- Time to Live (TTL) automatically expires cache entries, helping manage memory and prevent stale data.
- Effective cache invalidation strategies are critical to maintaining data freshness and system reliability.

## Amazon ElastiCache

Amazon ElastiCache is a fully managed, in-memory caching service that delivers sub-millisecond latency and high throughput for real-time applications.

- It supports Redis. Memcached and Valkey.
- Deployment options include serverless for ease of management and self-designed clusters for fine-grained control over configuration and scaling.
- Automatic and manual backup features ensure recoverability, and ElastiCache supports Redis and Memcached engines, each with distinct use cases and behaviors.

## Amazon MemoryDB

Ask

Amazon MemoryDB is a fully managed, in-memory primary database offering microsecond read and low-millisecond write latency.

- MemoryDB clusters consist of shards and nodes, with primary nodes handling writes and replicas handling reads across multiple AZs.
- It supports Multi-AZ replication with automatic failover, ensuring high availability and resilience against outages.
- Using online scaling, applications can scale MemoryDB clusters horizontally (adding shards) or vertically (upgrading node capacity) without downtime.
- Backup and restore capabilities, including periodic and on-demand snapshots, ensure point-in-time recovery from Amazon S3 storage.

## Test yourself

Take the quiz below to test what you've learned in the chapter.

**1. What is a major advantage of a Multi-AZ DB Cluster over a Multi-AZ Instance?**

A. Lower cost

B. Support for read scalability with read replicas

C. Smaller instance types

D. Magnetic storage optimization

1 / 10

Submit Answer

Ask